

1 Obstacle Avoidance

(c)

At SCP iteration i , the cost is already convex, so only the dynamics and obstacle constraints are replaced by affine approximations. The convex subproblem is

$$\begin{aligned} \min_{s_{\cdot|t}, u_{\cdot|t}} \quad & (s_{N|t} - s_{\text{goal}})^T P (s_{N|t} - s_{\text{goal}}) + \sum_{k=0}^{N-1} \left[(s_{k|t} - s_{\text{goal}})^T Q (s_{k|t} - s_{\text{goal}}) + u_{k|t}^T R u_{k|t} \right] \\ \text{s.t.} \quad & s_{0|t} = s(t) \\ & s_{k+1|t} = A_{f,k|t}^{(i)} s_{k|t} + B_{f,k|t}^{(i)} u_{k|t} + c_{f,k|t}^{(i)}, \quad k = 0, \dots, N-1 \\ & A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)} \geq 0, \quad k = 0, \dots, N \end{aligned}$$

where

$$\begin{aligned} A_{f,k|t}^{(i)} &= \left. \frac{\partial f}{\partial s} \right|_{(s_{k|t}^{(i)}, u_{k|t}^{(i)})} \\ B_{f,k|t}^{(i)} &= \left. \frac{\partial f}{\partial u} \right|_{(s_{k|t}^{(i)}, u_{k|t}^{(i)})} \\ c_{f,k|t}^{(i)} &= f(s_{k|t}^{(i)}, u_{k|t}^{(i)}) - A_{f,k|t}^{(i)} s_{k|t}^{(i)} - B_{f,k|t}^{(i)} u_{k|t}^{(i)} \\ A_{d,k|t}^{(i)} &= \left. \frac{\partial d}{\partial s} \right|_{s_{k|t}^{(i)}} \\ c_{d,k|t}^{(i)} &= d(s_{k|t}^{(i)}) - A_{d,k|t}^{(i)} s_{k|t}^{(i)} \end{aligned}$$

(d)

Each signed distance $d_j(s)$ is convex, so its first-order Taylor approximation is a global lower bound:

(g)

N	N_{SCP}	total time (s)	total control cost
5	5	20.30	6.93
2	5	18.80	11.18
5	2	11.81	4.43
15	2	14.49	4.50

$N = 5, N_{\text{SCP}} = 5$

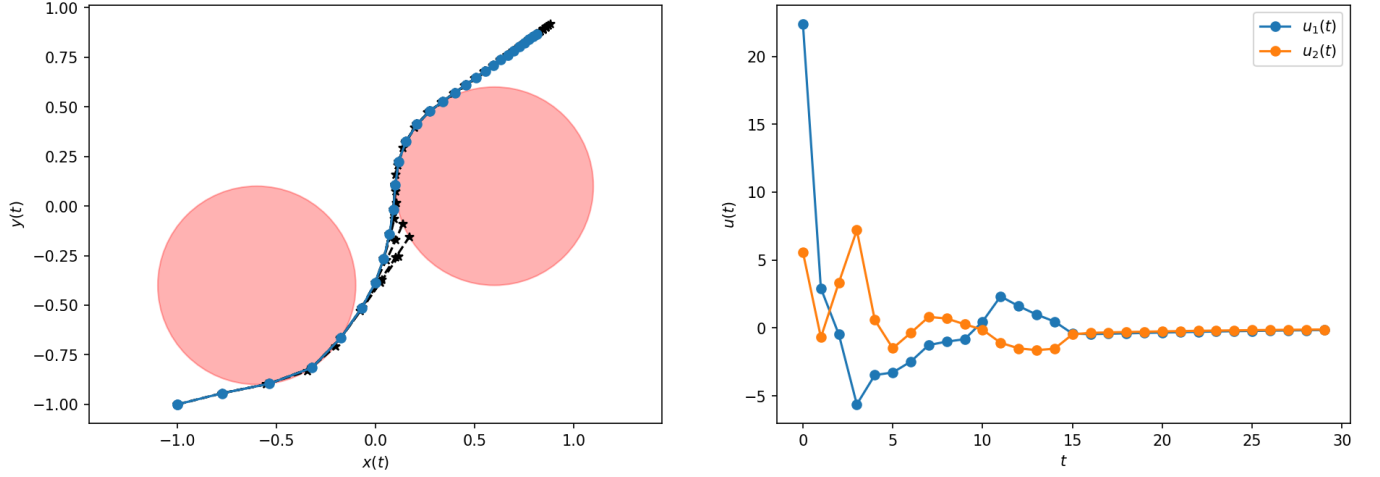


Figure 1: $N = 5, N_{\text{SCP}} = 5$

With $N = 5$ and $N_{\text{SCP}} = 5$, the trajectory avoids both obstacles and is reasonably smooth. This case takes the most time because each MPC solve uses more SCP iterations.

$N = 2, N_{\text{SCP}} = 5$

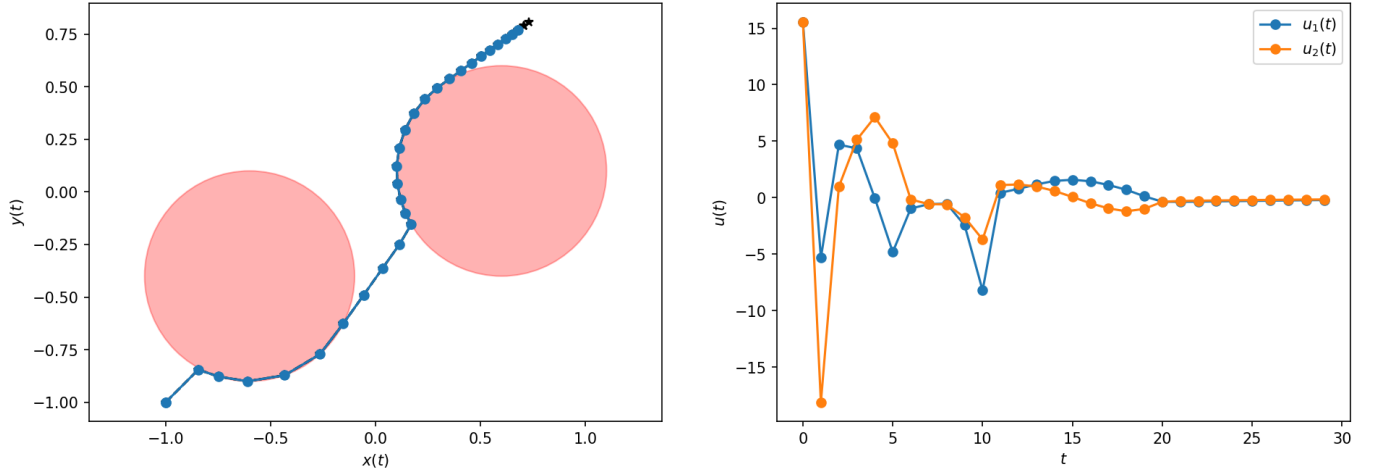


Figure 2: $N = 2, N_{\text{SCP}} = 5$

With $N = 2$, the controller is much more short-sighted. It still avoids the obstacles, but the path and controls are less smooth, and the total control cost is the largest.

$N = 5, N_{\text{SCP}} = 2$

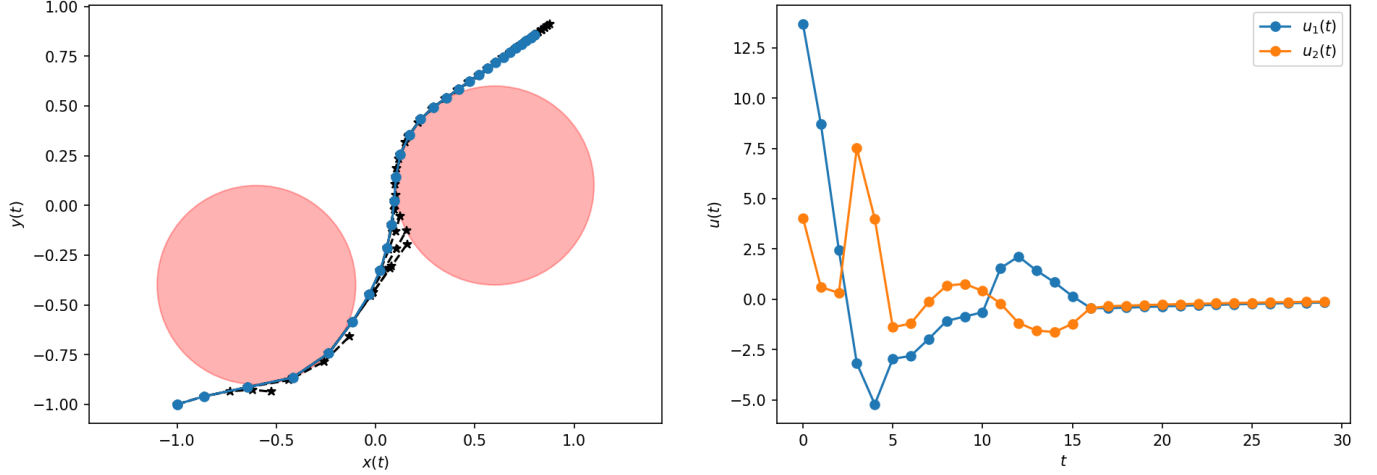


Figure 3: $N = 5, N_{\text{SCP}} = 2$

With $N = 5$ and only two SCP iterations, the computation time drops a lot while still giving a good trajectory. In this run it also gives the lowest control cost.

$N = 15, N_{\text{SCP}} = 2$

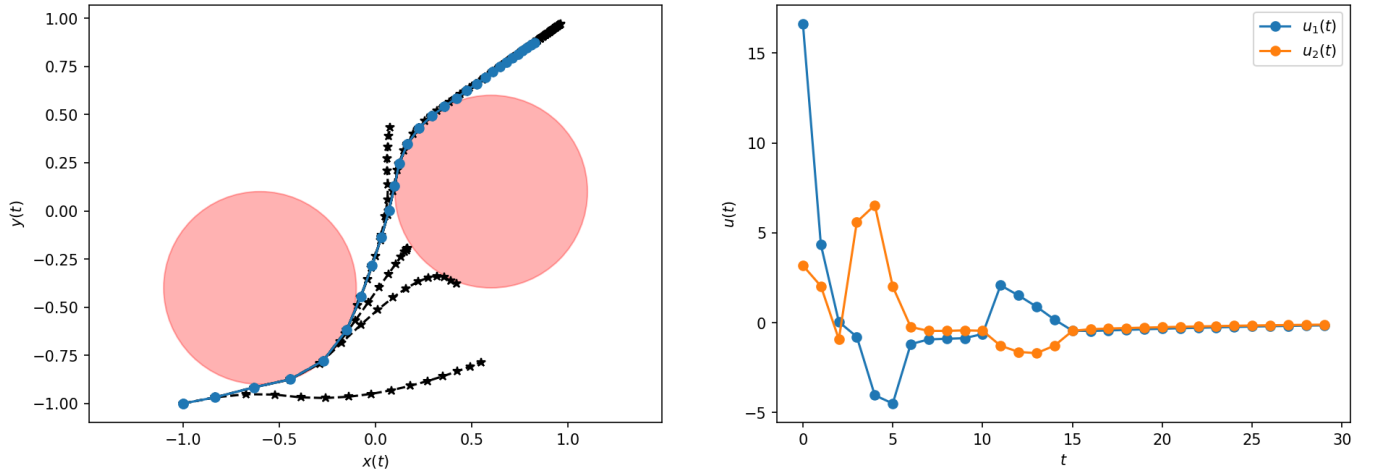


Figure 4: $N = 15, N_{\text{SCP}} = 2$

With $N = 15$, the planned trajectories are smoother because the controller can see farther ahead. The solve time is higher than $N = 5, N_{\text{SCP}} = 2$, but the control cost is still low.

Code

```

1 # PART (a): YOUR CODE BELOW #####
2 # INSTRUCTIONS: Compute the vector of signed distances to each obstacle.
3 d = jnp.linalg.norm(s[:2] - centers, axis=1) - radii
4 # END PART (a) #####
5
6 # PART (b) #####
7 # INSTRUCTIONS: Use JAX to affimize 'f' around '(s,u)' in two lines.

```

```

8 A,B = jax.jacobian(f, argnums=(0,1))(s,u)
9 c = f(s,u) - A @ s - B @ u
10 # END PART (b) #####
11
12 # PART (e): YOUR CODE BELOW #####
13 # INSTRUCTIONS: Construct the convex SCP sub-problem.
14
15 objective = 0.0
16 constraints = [ s_cvx[0,:] == s0 ]
17
18 for t in range(N):
19     objective += cvx.quad_form(s_cvx[t,:] - s_goal,Q) + cvx.quad_form(u_cvx[t,:], R)
20     constraints += [ Ad[t] @ s_cvx[t,:] + cd[t] >= 0,
21                     s_cvx[t+1] == Af[t] @ s_cvx[t,:] + Bf[t] @ u_cvx[t,:] + cf[t] ]
22
23 objective += cvx.quad_form(s_cvx[N,:] - s_goal,P)
24 constraints += [Ad[N] @ s_cvx[N, :] + cd[N] >= 0]
25
26 # END PART (e) #####
27
28 # PART (f): YOUR CODE BELOW #####
29
30 # Solve the MPC problem at time 't'
31 # Hint: You should call 'solve_obstacle_avoidance_scp' here
32 s_mpc[t], u_mpc[t] = solve_obstacle_avoidance_scp(f,d,s,s_goal,N,P,Q,R,eps,N_scp,s_init,
33             u_init)
34
35 # Push the state 's' forward in time with a closed-loop MPC input
36 # Hint: You should call 'f' here
37 s = np.array(f(s,u_mpc[t,0]))
38
39 # END PART (f) #####
40
41 # PART (g): CHANGE THE PARAMETERS BELOW #####
42
43 N = 15 # MPC horizon
44 N_scp = 2 # maximum number of SCP iterations
45
46 # END PART (g) #####

```

2 Widget Sales

(a)

```

1 for idx in shuffled_indices:
2     s = int(log["s"][idx])
3     a = int(log["a"][idx])
4     r = log["r"][idx]
5     s_next = int(log["s"][idx + 1])
6     a_idx = np.where(A == a)[0][0]
7
8     target = r + gamma * np.max(Q[s_next])
9     Q[s, a_idx] += alpha * (target - Q[s, a_idx])

```

(b)

```

1 Q_new = np.zeros_like(Q_vi)
2 for s in S:
3     for a_idx, a in enumerate(A):
4         expected_value = 0.0
5         for d, p in zip(D, P):
6             s_next = transition(s, a, d)
7             expected_value += p * (reward(s, a, d) + gamma * np.max(Q_vi[s_next]))
8         Q_new[s, a_idx] = expected_value
9 Q_vi = Q_new

```

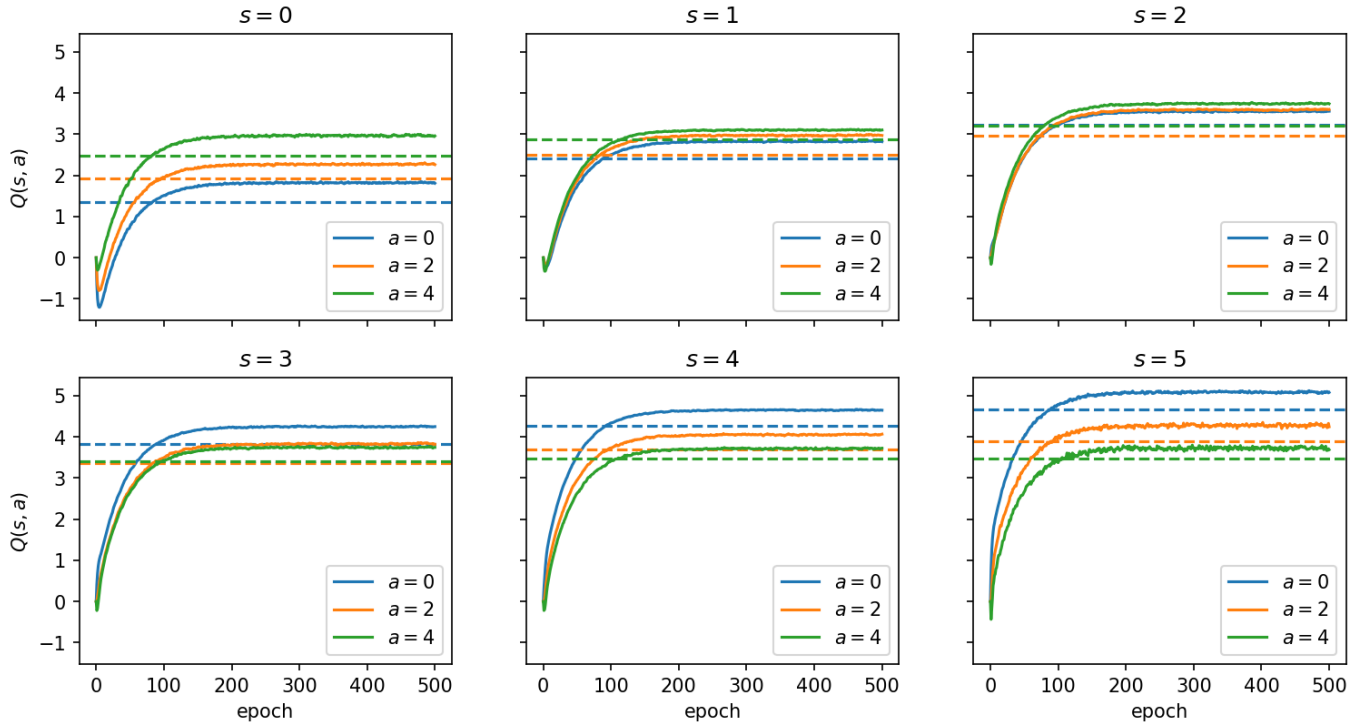


Figure 5: Q-learning estimates vs value-iteration Q-values

The Q-learning values approach the value-iteration values but do not match exactly. This is expected because Q-learning only sees a finite logged dataset generated by a random policy, while value iteration uses the exact demand model and averages over all possible demands.

(c)

```
1 T = 5 * 365
2
3 a_opt_ql = A[np.argmax(Q, axis=1)]
4 a_opt_vi = A[np.argmax(Q_vi, axis=1)]
5
6 policy_ql = lambda s: a_opt_ql[int(s)]
7 policy_vi = lambda s: a_opt_vi[int(s)]
8
9 _, _, r_ql = simulate(np.random.default_rng(seed), policy_ql, T)
10 _, _, r_vi = simulate(np.random.default_rng(seed), policy_vi, T)
11
12 profit_ql = np.cumsum(r_ql)
13 profit_vi = np.cumsum(r_vi)
```

The optimal policies are:

```
1 Optimal policy (Q-learning):      [4 4 4 0 0 0]
2 Optimal policy (value iteration): [4 4 0 0 0 0]
```

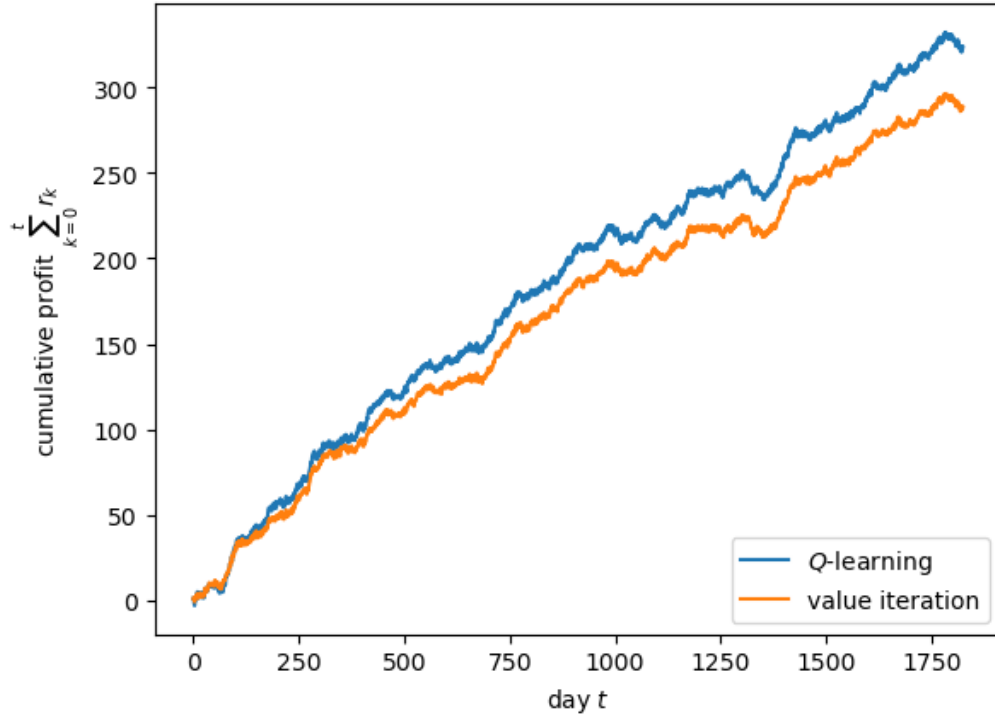


Figure 6: Cumulative profits over five years

The Q-learning policy ended with cumulative profit ~ 320 , while the value-iteration policy ended with cumulative profit ~ 280 . The Q-learning policy orders more aggressively at $s = 2$, which happens to do better on this sampled demand sequence. Value iteration is still the cleaner policy in expectation because it is computed from the full demand model, while the Q-learning policy is shaped by finite logged data.

3 Cart-pole Balance via Reinforcement Learning

(a)

(i) build_network

```
1 return torch.nn.Sequential(  
2     torch.nn.Linear(self.state_dim, self.hidden_dim),  
3     torch.nn.ReLU(),  
4     torch.nn.Linear(self.hidden_dim, self.hidden_dim),  
5     torch.nn.ReLU(),  
6     torch.nn.Linear(self.hidden_dim, self.action_dim),  
7 )
```

(ii) policy

```
1 if train and random.random() < self.eps_threshold():  
2     return torch.randint(self.action_dim, (1,), device=self.device)  
3  
4 with torch.no_grad():  
5     q_values = self.policy_network(state)  
6     return torch.argmax(q_values, dim=-1)
```

(iii) train

```
1 optimizer = torch.optim.AdamW(self.policy_network.parameters(), lr=self.learning_rate)  
2 loss_fn = torch.nn.MSELoss()  
3  
4 for _ in tqdm(range(num_episodes)):  
5     obs, info = env.reset()  
6     state = torch.tensor(obs, dtype=torch.float32).unsqueeze(0).to(self.device)  
7     terminated, truncated = False, False  
8  
9     while not terminated and not truncated:  
10         action = self.policy(state, train=True)  
11         obs, reward, terminated, truncated, info = env.step(action.item())  
12  
13         reward = torch.tensor([reward], dtype=torch.float32).to(self.device)  
14         next_state = None  
15         if not terminated and not truncated:  
16             next_state = torch.tensor(obs, dtype=torch.float32).unsqueeze(0).to(self.  
17                 device)  
18  
19         self.buffer.append(Transition(state, action, reward, next_state))  
20         state = next_state  
21  
22         if len(self.buffer) > self.batch_size:  
23             states, actions, targets = self.sample_buffer()  
24             q_values = self.policy_network(states).gather(1, actions)  
25             loss = loss_fn(q_values, targets)  
26  
27             optimizer.zero_grad()  
28             loss.backward()  
29             torch.nn.utils.clip_grad_value_(self.policy_network.parameters(), 100)  
30             optimizer.step()
```

(iv) Training results

Using 10 evaluation rollouts, the randomly initialized DQN policy had

```
1 --- Reward Statistics ---  
2 Average: 9.4
```

```
3 Standard deviation: 0.9165151389911681
4 Minimum: 8.0
5 Maximum: 11.0
```

After 100 training episodes, the evaluated policy had

```
1 --- Reward Statistics ---
2 Average: 9.7
3 Standard deviation: 0.6403124237432849
4 Minimum: 9.0
5 Maximum: 11.0
```

(b)

(i) Naive policy gradient

```
1 # 1) Naive REINFORCE
2 discounted_return = torch.tensor(
3     sum((self.gamma ** t) * rewards[t] for t in range(len(rewards))),
4     dtype=torch.float,
5 )
6 naive_loss = -torch.stack(
7     [log_prob * discounted_return for log_prob in log_probs]
8 ).sum()
9
10 # 2) REINFORCE with causality trick
11 returns = []
12 running_return = 0.0
13 for reward in reversed(rewards):
14     running_return = reward + self.gamma * running_return
15     returns.insert(0, running_return)
16 returns = torch.tensor(returns, dtype=torch.float)
17 causal_loss = -torch.stack(
18     [log_prob * ret for log_prob, ret in zip(log_probs, returns)]
19 ).sum()
20
21 # 3) REINFORCE with causality trick and baseline to "center" the returns
22 baseline = returns.mean()
23 baseline_loss = -torch.stack(
24     [log_prob * (ret - baseline) for log_prob, ret in zip(log_probs, returns)]
25 ).sum()
26
27 if self.gradient_variant == "naive":
28     loss = naive_loss
29 elif self.gradient_variant == "causal":
30     loss = causal_loss
31 else:
32     loss = baseline_loss
```

(iv) Training results

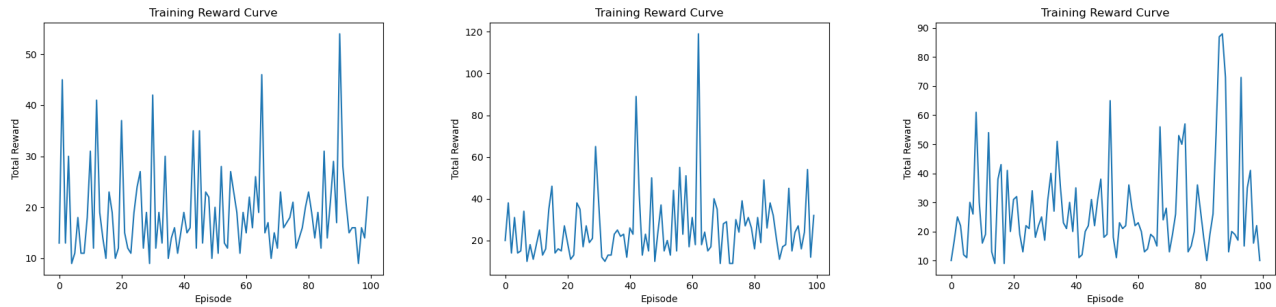


Figure 7: REINFORCE reward curves for naive, causal, and baseline variants

```

1 --- Reward Statistics ---
2 Average: 152.7
3 Standard deviation: 54.30478800253251
4 Minimum: 86.0
5 Maximum: 248.0

```

Compared with the DQN run above, REINFORCE performed much better after training. The reward curve is still noisy because policy-gradient updates use sampled trajectories directly, but the baseline estimator helps reduce variance by centering the returns.